IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received June 23, 2021, accepted July 22, 2021, date of publication July 26, 2021, date of current version August 2, 2021.

Digital Object Identifier 10.1109/ACCESS.2021.3100082

Zeus: Improving Resource Efficiency via Workload Colocation for Massive Kubernetes Clusters

XIAOLONG ZHANG[1,2], LANQING LI[2], YUAN WANG[2], E. CHEN[2], AND LIDAN SHOU[1], (Member, IEEE)

[1]College of Computer Science and Technology, Zhejiang University, Hangzhou 310023, China
[2]NetEase (Hangzhou) Network Company Ltd., Hangzhou 310052, China

Corresponding author: Xiaolong Zhang (hzzhangxl@corp.netease.com)

ABSTRACT With the popularity of container-based microservices and cloud-native architectures, Kubernetes has established itself as the de facto standard for container orchestration. Kubernetes is known for its advantage in easy deployment and operations for applications; however, it suffers from low resource utilization, incurring high server provisioning and operational costs, due to following reasons. First, it is common practice for latency-sensitive services to be over-provisioned for the peak load: Kubernetes might consider such peak-load provisioning as constant, even though the actual resource utilization is low. Second, the isolation between different containers is poor and cannot prevent performance degradation when best-effort jobs and latency-sensitive services run together. Users may have to run these two classes of workloads on separate machines to avoid interference, resulting in higher provisioning costs. This paper presents a highly scalable cluster scheduling system named Zeus, which is designed based on Kubernetes extension mechanisms. Zeus achieves safe colocation of best-effort jobs and latency-sensitive services. Furthermore, Zeus schedules best-effort jobs based on the real server utilization and can adaptively allocate resources between the two classes of workloads. In addition, Zeus enhances container isolation by coordinating software and hardware isolation features. As a result, Zeus can effectively improve the resource utilization of Kubernetes clusters. We discuss the design and implementation of Zeus and evaluate its effectiveness using latency-sensitive services and best-effort jobs in a massive production environment. The results show that by colocating latency-sensitive services with best-effort jobs, Zeus can increase the average CPU utilization from 15% to 60% without SLO violations.

INDEX TERMS Big data, Kubernetes, performance isolation, resource efficiency, workload colocation.

I. INTRODUCTION

Cloud computing and big data have become the mainstream of next-generation information technology, and their rapid development has become an important driving force for large-scale data centers. New server acquisition accounts for 50%-70% of the total cost of ownership (TCO) for modern data centers [1], but several studies show that the average resource utilization of data center servers is low, approximately 10-20% [2]–[6], which results in high operational cost and substantial resource wastage.

Moreover, the virtualization technology of the data center is gradually being transitioned from virtual machines to containers [7]. As a lightweight virtualization technology, containers can greatly improve the efficiency of application development and deployment with no virtualization overhead [8]–[10]. Kubernetes has now become the de facto standard platform for managing container clusters. Additionally, Kubernetes is widely used on public clouds and in enterprise IT systems, and many operators are using Kubernetes to run and manage workloads outside of web applications and microservices. Typical scenarios include training jobs for machine learning, high-performance computing jobs, and conventional big data processing jobs [11]–[14]. Spark officially introduced native support for Kubernetes with version v2.3.0.

However, batch jobs compete for shared-resources with user-facing latency-sensitive services when they run together,

The associate editor coordinating the review of this manuscript and approving it for publication was Chi-Tsun Cheng.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

which might degrade the end-user experience. Thus, the two classes of workloads are typically run on separate machines when using Kubernetes, which can prevent performance interference but leads to underutilization. As shown in Fig. 1, the mean CPU utilization of a production Kubernetes cluster is 20%, but CPU request reaches 80%. Latency-sensitive services commonly request more resources than they should due to a bad configuration, often using 100% of the available CPU or memory resources. However, there remain abundant idle resources that can be reused.

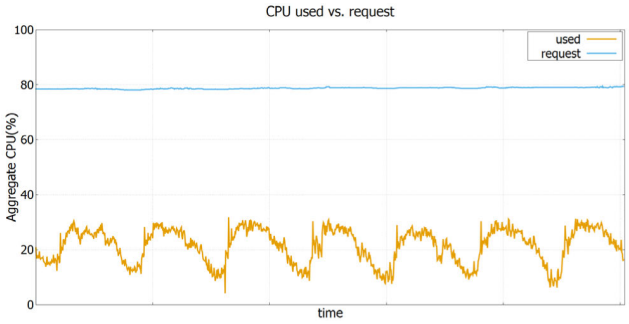


FIGURE 1. The poor CPU utilization of an over 5000-node production Kubernetes cluster that runs latency-sensitive services only.

Colocating batch jobs alongside latency-sensitive services is an ideal solution to improve the efficiency of data centers [2], [3], [15]–[18]. Batch jobs can use idle resources that have been requested but are not currently used by services. As a popular production-grade container orchestration framework, Kubernetes provides convenient workload colocation, but its native scheduling and resource isolation mechanisms cannot support safe workload colocation to improve resource efficiency for two reasons:

- Static resource scheduling: Kubernetes does not consider the actual CPU and memory utilization of a node when making scheduling decisions; it factors in only the resource descriptions provided by the developer or operator. Kubernetes might consider a node as being fully utilized if all the resources are requested by services, even though the actual resource utilization of the node is in fact very low. Latency-sensitive services are always over-provisioned for peak load, which only lasts a very short duration. Thus, a substantial disparity exists between the requested resources and the actual resource utilization.
- Limited container isolation: The isolation between different Kubernetes containers is limited; the CPU and memory isolation effectiveness is inadequate. In addition, other shared resources, such as last-level cache (LLC) and network bandwidth, are not considered. Moreover, latency-sensitive services may suffer from unpredictable performance degradation when run together with batch jobs.

The main challenge of achieving workload colocation in Kubernetes is to maximize server resource utilization while meeting the service-level objectives (SLOs) of latency-sensitive services and reducing the batch job completion time.

Diverse workloads with different target latency exist in production environments. Existing solutions [15], [16], [18], [19] query real-time performance metrics of services and adjust the resource allocation accordingly. However, assuming that all workloads target SLOs and that real-time service-level performance metrics are known is not realistic. In addition, it is very expensive to conduct all possible colocation scenario experiments.

This article describes Zeus, a cluster scheduling system that maximizes resource utilization for Kubernetes clusters via workload colocation while meeting the SLOs of latency-sensitive services. We share our best practices on improving resource efficiency for massive Kubernetes clusters in a production environment.

We design a dynamic scheduling algorithm based on server resource utilization for batch jobs. A monitoring agent runs on each server to report server resource usage to the scheduler in real time, and the scheduler uses a weighted time sliding window dynamic algorithm to calculate and select the ''optimal'' candidate scheduling node according to the historical load information of nodes and services. This approach can reduce resource competition between services and jobs, prevent performance degradation of services and maximize job throughput. Meanwhile, we design a rescheduling and balancing algorithm for batch jobs to avoid job starvation and severe resource competition since the previous scheduling decisions might become obsolete and unreasonable considering the load fluctuation of all nodes.

The resource isolation between containers in Kubernetes is very poor and cannot effectively handle colocation of latency-sensitive services and batch jobs [20], [21]. We propose a multidimensional resource isolation strategy that coordinates the server's software and hardware characteristics and takes many shared resources, such as CPU, memory, LLC, network bandwidth, and disk, into consideration. The isolation of memory bandwidth and LLC is achieved via hardware; the isolation of CPU and network bandwidth is achieved via software. In the CPU dimension, we run services and jobs on two separate cpusets. Latency-sensitive services have higher CPU priority, and we dynamically adjust the configuration of the cpuset based on the resource requirements of the services.

We develop the cluster scheduling system Zeus based on the extension mechanism of Kubernetes. We do not change any Kubernetes in-tree code; thus, Zeus can easily be deployed to any standard Kubernetes cluster. Moreover, we do not make any assumptions about services, hardware and OS: we treat them as black-box to promote the wide deployment of this system.

To evaluate Zeus, we select Spark as batch jobs to collocate with representative latency-sensitive services in both test and production environments. The results show that Zeus can effectively handle resource scheduling and isolation in colocation scenarios. It can improve server resource utilization

from 15% to 60% while meeting the SLOs of services and minimizing job completion time.

Our main contributions are listed as follows:

- We analyze the root cause of low resource utilization in Kubernetes.
- We propose a framework called Zeus to address the low resource utilization problem: (1) we design a dynamic scheduling algorithm based on the actual resource utilization of servers and use a weighted time sliding window-based algorithm to perform dynamic scheduling and rescheduling for batch jobs; (2) we design a multidimensional resource isolation strategy for coordinating software and hardware mechanisms to achieve resource isolation on CPU, memory, cache, and network, effectively reducing the performance interference between services and jobs.
- We implement the Zeus cluster scheduling framework based on Kubernetes. In addition, we evaluate its effectiveness in improving resource efficiency on a massive Kubernetes cluster in a production environment.

To the best of our knowledge, Zeus is the first open-source solution reported to effectively improve resource efficiency for massive Kubernetes clusters with complex and diverse workloads.

II. KUBERNETES NATIVE FEATURE

This chapter introduces the container orchestration system Kubernetes. First, we describe the architecture and basic concepts of Kubernetes. Then, we introduce the Kubernetes scheduling framework and its native disadvantages. Finally, we present the Kubernetes container isolation mechanism and discuss its limitations.

A. KUBERNETES ARCHITECTURE

The architecture of Kubernetes is shown in Fig. 2. Kubernetes follows the Master-Slave model, and nodes are divided into two roles: Master and Worker. The Master is the control node of Kubernetes and is responsible for the management and control of the entire cluster, i.e., the brain of the cluster. The following key components run on the Master node:

- Etcd: Distributed key-value database, which is used to coordinate resources and configuration data of the cluster between different components.
- Kube-apiserver: Provides the Restful interface, which is the only entry point to the cluster operations.
- Kube-controller-manager: A daemon that embeds the core control loops shipped with Kubernetes. In applications of robotics and automation, a control loop is a nonterminating loop that regulates the state of the system.
- Kube-scheduler: Continuously selects pending Pods from queue and assigns them to nodes with sufficient requested resources. A Pod is a group of one or more containers with shared storage and network resources and a specification for how to run the containers; it is the most basic object that can be created, scheduled and deployed.

The other machines in a Kubernetes cluster are called Worker nodes, which run user applications. Each Worker node is assigned multiple workloads by the Master. When a Worker node fails, its workload is automatically migrated to other healthy Worker nodes. Node components run on every node as agents to monitor and manage running Pods:

- Kubelet: The primary "node agent" responsible for monitoring and managing all the containers on a node.
- Kube-proxy: A network proxy that enables the Kubernetes service abstraction.
- Container runtime: Responsible for the creation and management of containers.

Pods can be categorized into three quality of service (QoS) classes, which is inferred from requests and limits in pod specification [11]:

- Best-effort (lowest priority): Containers in the Pod have not set the request and limit of CPU and memory.
- Guaranteed (highest priority): All containers in the Pod have set the request and limit of CPU and memory, and the request value is equal to the limit value.
- Burstable: The rest of the Pods are burstable.

Kubernetes uses QoS classes to make decisions about scheduling and evicting Pods. Guaranteed Pods are considered highest priority and have the highest resource guarantee. Burstable Pods have some resource guarantee that the request manifest specifies and can use more resources when available. Best-effort Pods are treated as lowest priority and do not have any resource guarantee. The oom_score_adj value of Best-effort Pods is the largest, which means that Best-effort Pods are killed first when the OS runs out of memory. When the load of noncompressible resources such as memory and disk on a node is too high to affect the stability of node, kubelet evicts the above Pods to ensure the stability of the node. The order of eviction is Best-effort, Burstable, Guaranteed.

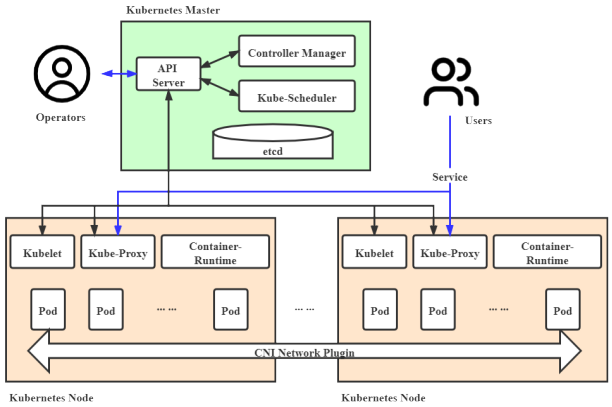


FIGURE 2. Kubernetes architecture.

In addition, users can configure priority classes to indicate the relative priority of Pods. Furthermore, the Kubernetes scheduler takes into consideration the priority of a Pod and can even preempt (remove) Pods with lower priority to make room on a worker node for higher priority Pods.

B. KUBERNETES SCHEDULING

Kube-scheduler is a core component that affects resource utilization and system performance. The newly created Pods are added to a Pending Queue. Kube-scheduler continuously fetches Pods from the queue one by one and schedules them to the best node with sufficient resources based on the Pod's resource request and other conditions. For each pending Pod, a two-phase operation is designed:

(1) Filter: Finds the set of nodes where it is feasible to schedule the Pod.
(2) Priority: Ranks the filtered nodes to choose the most suitable Pod placement. The scheduler assigns a score to each node that survived filtering based on the active scoring rules. Kube-scheduler then assigns the Pod to the node with the highest ranking.

In the Filter phase, when Kube-scheduler judges whether the idle resources on the node are sufficient, it does not refer to the actual load of the node but considers the static configuration resource request of Pods.

In the Priority phase, Kube-scheduler adopts the LeastRequested algorithm. It selects the node with the smallest amount of resource requests first and tries to balance the load between different nodes based on the following equations:

$$\text{score} = \frac{memScore + cpuScore}{2} * 10 \tag{1}$$

$$\text{memScore} = \frac{memCap - memReq}{memCap} \tag{2}$$

$$\text{cpuScore} = \frac{cpuCap - cpuReq}{cpuCap} \tag{3}$$

memCap represents the memory capacity of the node, memReq represents total amount of memory requested by all Pods on the node, cpuCap represents the CPU capacity of the node, and cpuReq represents total amount of CPU resources requested by all Pods on the node.

Kube-scheduler uses a static scheduling mechanism in both the Filter phase and Priority phase. This mechanism has the advantage of simple implementation and can guarantee requested resources of Pods, but it easily leads to low resource utilization and unbalanced loads among nodes. The static policy is either too conservative (low resource utilization) or overly optimistic (leading to SLO violations) [16]. Latency-sensitive services are always over-provisioned for peak load, and Kubernetes might consider a server as being fully utilized if all resources are requested by applications, even though the actual resource utilization of the server is in fact very low. As shown in Fig. 3(a), the actual utilization of node1 is lower than that of node2, but the request level is higher than that of node2; thus, Kubernetes prefers to schedule Pods to node2. However, as shown in Fig. 3(b), node1 with lower actual utilization is a better choice.

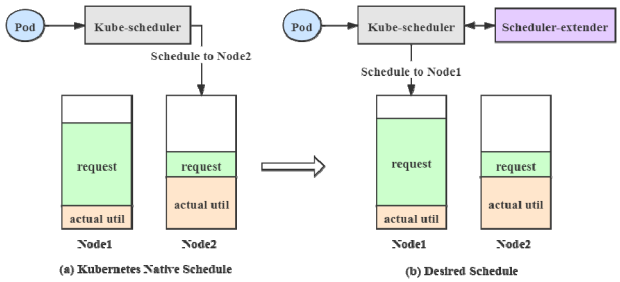


FIGURE 3. Kubernetes native schedule vs. desired schedule.

C. KUBERNETES CONTAINER ISOLATION

When multiple containers are running on the same node, they compete for the shared resources on the machine, such as CPU, memory, network, and IO. Kubernetes natively supports the scheduling of only CPU and memory resources, so it isolates these two types of resources. The CGroups manager in Kubelet configures the CGroups parameters according to the Pods' resource requests and resource limits to achieve a certain degree of resource isolation.

For CPU resources, requests.cpu in the Pods corresponds to the cpu.shares configuration in CPU CGroups, which defines a relative weight of that container for the CPU time. limits.cpu maps to cpu.cfs_quota_us in CGroups, which limits the CPU time of a container's processes. CPU is a compressible resource. Pods can obtain at least the requested resources, and if Pods use up all the time specified by the quota, they are throttled for the remainder of the specified period and are not allowed to run until the next period.

For memory resources, limits.memory in Pods maps to memory.limit_in_bytes in CGroups, which means processes in the container are killed by the kernel if they hit the specified memory usage (OOMKilled).

However, the aforementioned isolation mechanism is more limited than might be expected and cannot ensure that latency-sensitive services are free from performance degradation. When services and jobs run on the same physical CPU core or hyper-thread, Linux Completely Fair Scheduler (CFS) cannot guarantee that batch jobs will not affect the SLOs of latency-sensitive services [4], [16], [22]–[24]. CGroups memory.limit_in_bytes limits the upper bound of only memory capacity usage, not the memory bandwidth. In addition, the physical machine resources shared between the containers also include network bandwidth, IO bandwidth, cache, etc. Container abstraction provides less isolation, and sharing resources might lead to unpredictable performance interference.

In this paper, we address the limitations of Kubernetes native schedule and achieve multidimensional resource isolation on CPU, memory, cache, and network, effectively reducing the performance interference between services and jobs.

III. ZEUS SYSTEM DESIGN

The main design objectives of Zeus are summarized as follows:

- Easy to deploy widely: The cost of data centers and workload transformation is very large and not realistic in the actual production environment, so Zeus must treat the workloads as a black box and be able to be deployed to any standard Kubernetes cluster.
- Maximize resource utilization through dynamic scheduling and dynamic resource allocation: Static allocation of resources cannot take full advantage of idle resources during off-peak times. Through dynamic scheduling and dynamic resource allocation mechanisms, Zeus enables batch jobs to use temporarily unused resources of latency-sensitive services to maximize resource utilization.
- Ensure batch jobs will not affect the SLOs of latency-sensitive services and improve the execution efficiency of batch jobs as much as possible to shorten the completion time.

A. OVERVIEW AND ARCHITECTURE

We define a new API object, NodeProfile, based on the Kubernetes Custom Resource Definition to describe the resource profile of the node. We introduce two extended resources, namely, colocation/cpu and colocation/memory, which represent idle resources that batch jobs can use.

Fig. 4 describes the system architecture of Zeus; all components are developed based on the Kubernetes extension mechanism.

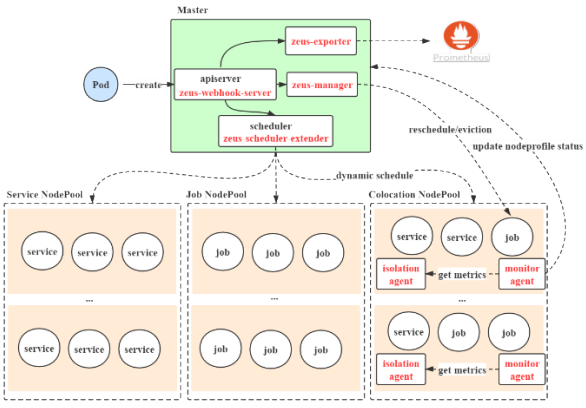


FIGURE 4. Zeus system structure.

In addition to the existing components of Kubernetes, the control plane of Zeus consists of four new components:

- Zeus-webhook-server: The webhook server intercepts requests to the Kube-apiserver prior to persistence of the object; it may be "validating", "mutating", or both.
- Zeus-manager: Zeus-manager is developed based on the Kubernetes operator mode. It realizes the life cycle management of NodeProfile resources, rescheduling of batch jobs, etc.
- Zeus-scheduler-extender: This component is developed based on the Kubernetes scheduler http extender mechanism to implement dynamic scheduling of batch jobs.
- Zeus-exporter: The exporter collects the resource usage metrics of the Kubernetes cluster.

Additionally, Zeus runs two agents on each node:

- Zeus-monitor-agent: The monitor agent collects real-time metrics of resource usage and reports to NodeProfile objects.
- Zeus-isolation-agent: The isolation agent achieves multidimensional resource isolation on CPU, memory, cache, and network. It regularly queries node resource portraits from the Zeus-monitor-agent and applies isolation rules accordingly.

As shown in Fig. 5, when a user wants to create batch jobs, he or she needs only to set the Pods label colocation.netease.com/workload-type = batch-job, and containers still request native CPU and memory resources. As shown in Fig. 6, the Zeus-webhook-server intercepts the Pods creation request and mutates fields in the Pods. Zeus-webhook-server converts native CPU/memory resource requests into colocation/cpu and colocation/memory for batch jobs, so batch jobs are also categorized as Best-effort QoS class. In addition, when the priority of a Pod is set to −100, the priority is lower than that of latency-sensitive services, so when Kube-scheduler schedules the Pod, it will schedule latency-sensitive services first, and then schedule batch jobs.

```yaml
apiVersion: v1
kind: Pod
metadata:
    name: example-job
    labels:
        colocation.netease.com/workload-type: batch-job
spec:
    resources:
        requests:
            cpu: 2
            memory: 4Gi
```

FIGURE 5. Users set specified Pod labels when creating batch jobs.

B. COLOCATION RESOURCE CAPACITY

Zeus schedules batch jobs based on colocation/cpu and colocation/memory resources. First, we need to determine the total capacity of colocation resources for each node.

By default, Zeus-monitor-agent calculates the capacity of colocation/cpu and colocation/memory every 10 seconds, and reports the results to the control plane. The calculation is shown in equations (4) and (5):

$$\text{cccpu}_i = \max(tc * acpu_i - scu_i, rc * acpu_i) \quad (4)$$

$$\text{ccmem}_i = \max(tm * amem_i - smu_i, rm * amem_i) \quad (5)$$

The variable definitions and system parameters are given in Tables 1 and 2.

On each node, Zeus adapts rapidly to allocate different amounts of resources to batch jobs. If latency-sensitive

```yaml
apiVersion: v1
kind: Pod
metadata:
    name: example-job
    labels:
        colocation.netease.com/workload-type: batch-job
spec:
    priority: -100
    resources:
        requests:
            colocation/cpu: 2
            colocation/memory: 4Gi
```

FIGURE 6. Zeus-webhook-server intercepts the Pods creation request and mutates some fields.

TABLE 1. Variable definitions.

Symbol	Meaning
$cccpu_i$	The capacity of colocation/cpu of node i
$ccmem_i$	The capacity of colocation/memory of node i
$accpu_i$	The colocation/cpu requested by all batch jobs on node i
$acmem_i$	The colocation/memory requested by all batch jobs on node i
$uccpu_i$	The actual colocation/cpu used by all batch jobs on node i
$ucmem_i$	The actual colocation/memory used by all batch jobs on node i
$fccpu_i$	The total amount of colocation/cpu currently available for batch jobs on node i
$fcmem_i$	The total amount of colocation/memory currently available for batch jobs on node i
$rccpu_{pod}$	The amount of colocation/cpu resources requested by the Pod
$rcmem_{pod}$	The amount of colocation/memory resources requested by the Pod
$candidateNodes$	All nodes that meet the filter conditions
$acpu_i$	The total amount allocatable CPU of node i
$amem_i$	The total amount allocatable memory of node i
scu_i	The CPU actually used by the latency-sensitive services on node i
smu_i	The actual memory used by the latency-sensitive services on node i

The variable definitions and system parameters are given in Tables I and II.

services use more resources, fewer resources are allocated to batch jobs; if latency-sensitive services use less resources, then more resources are allocated to batch jobs. Additionally, to avoid starvation for batch jobs, we guarantee a minimum amount of allocatable resources for batch jobs, using $rc * acpu$ and $rm * amem$. This calculation method is simple but effective. It does not make any assumptions about workloads, and it does not require the workloads to provide the SLOs and metric interfaces. The total amount of resources allocated to batch jobs depends on the target resource utilization and actual resource usage of latency-sensitive services.

C. DYNAMIC SCHEDULE AND RESCHEDULE

Zeus takes the actual CPU and memory utilization of each node into consideration when making scheduling decisions for batch jobs. Batch jobs can effectively borrow redundant

TABLE 2. System parameters.

Symbol	Meaning	Value Ranges
rc	The minimum percentage of CPU resources allocated to batch jobs	[0,100%]
rm	The minimum percentage of memory resources allocated to batch jobs	[0,100%]
tc	The expected CPU utilization	(0,100%]
tm	The expected memory utilization	(0,100%]
decay	The weight of resource usage metrics is halved every 'decay' duration	$(0, \infty)$
w_{cpu}	The weight of CPU usage metrics when calculating the score	(0,1]
w_{mem}	The weight of memory usage metrics when calculating the score	(0,1]
w[t]	The weight of resource usage metrics at time t when calculating scores for a node	(0,1]
SCORE	The preference for a node when scheduling Pods. Finally, we reduce the value range to (0,10]	(0,10]
hungerDegree	The degree of hunger of batch jobs.	$(0, \infty)$

resources from latency-sensitive services that are currently idle. With the dynamic scheduling technology, we can achieve load balance among nodes, and minimize resource competition between services and jobs. The rescheduling mechanism can avoid job starvation and minimize resource contention since the previous scheduling decisions might become obsolete and unreasonable considering the load fluctuation of all nodes.

New scheduling rules are added to Kubernetes by means of a standardized procedure: implementing a ''scheduler extender'' process that the standard Kubernetes scheduler calls as a final pass when making scheduling decisions. We implement Filter and Priority functions in the scheduler extender, and they are centralized instead of distributed. Filter finds all candidate nodes that meet all specific requirements of the Pods, and then Priority scores all candidate nodes. Finally, the scheduler schedules Pods to the node with the highest score.

1) FILTER

The Filter algorithm is shown in the pseudo code of Algorithm 1. For each node, the total resource requests of scheduled batch jobs are subtracted from the total capacity of colocation resources. If the remaining schedulable resources are greater than the resource request of the Pods to be scheduled, the filter conditions are met. The time complexity of Algorithm 1 is O(n), where n is the number of nodes. We can start multiple working threads to accelerate the processing and invoke the filter of each node in parallel.

2) PRIORITY

Zeus-monitor-agent reports resource usage (CPU, memory, etc.) of the node every 10 seconds, including the resource

Algorithm 1 Filtering of Zeus-Scheduler-Extender

Input: *pod, nodes*
Output: *candidateNodes*
for each $n \in nodes$ **do**
  $fccpu_n = cccpu_n - \max(uccpu_n, accpu_n)$
  $fmem_n = ccmem_n - \max(ucmem_n, acmem_n)$
  if $\mathbf{fccpu}_n < rccpu_{pod}$ **then**
    continue
  end if
  if $\mathbf{fcmem}_n < rcmem_{pod}$ **then**
    continue
  end if
  insert n into *candidateNodes*
end for
return *candidateNodes*

usage at the node level, the aggregated resource usage of the latency-sensitive services on the node, and the overall aggregated usage of the batch jobs on the node.

Define $score_i$ as the score of node i based on the current load data; w_{cpu} and w_{mem}, respectively, represent the weight of CPU and memory resources when calculating the score:

$$\text{score}_i(cpu) = cccpu_i - \max(uccpu_i, accpu_i) \tag{6}$$

$$\text{score}_i(mem) = ccmem_i - \max(ucmem_i, acmem_i) \tag{7}$$

$$\text{score}_i = w_{cpu} * score_i(cpu) + w_{mem} * score_i(mem) \tag{8}$$

The lower the node load is, the more colocation resources available and the higher the score. Zeus preferentially schedules batch jobs to the node with the highest score. To avoid starvation and reduce the job completion time, Zeus takes the historical 24 hours load information into consideration, uses the moving weighted average algorithm to calculate the score, and defines the weight w[t] as the weight value from the current time t:

$$w[t] = 2^{-\frac{t}{decay}} \tag{9}$$

The closer t is to the current time, the higher the $w[t]$, which decays exponentially. The half-life of the weight is 6 hours, that is, the weight value is halved every 6 hours.

$$SCORE_i = \frac{\sum_{t=0}^{24h} w[t] * score_i[t]}{\sum_{t=0}^{24h} w[t]} \tag{10}$$

$score_i[t]$ represents the score of node i calculated based on the load data of time t; $SCORE_i$ represents the final score of node i. Zeus preferentially schedules batch jobs to the node with the highest $SCORE_i$.

The time complexity of the Priority algorithm is O(m∗n), where m is the number of resource usage sampling points and n is the number of nodes. We can start multiple working threads to accelerate the processing, and the scoring of each node is invoked in parallel.

3) RESCHEDULE

With the creation and destruction of Pods, the load on different nodes varies and gradually becomes unbalanced. Batch jobs on some nodes have sufficient idle resources, while batch jobs on other nodes are hungry and have low execution efficiency. Therefore, we implement the Reschedule mechanism. Specifically, we denote the degree of hunger of batch jobs on the node by *hungerDegree*:

$$\text{hunger}Degree_i = \frac{accpu_i}{cccpu_i} \tag{11}$$

Under normal circumstances, *hungerDegree* is less than or equal to 1. If *hungerDegree* is greater than 1, the batch job is hungry. The larger the value, the hungrier it is. If the *hungerDegree* of a node is greater than the threshold for a long time while other nodes have sufficient colocation resources, the batch jobs will be rescheduled to other idle nodes one by one according to the job priority from low to high until *hungerDegree* is less than or equal to 1. The Reschedule mechanism balances the load among nodes, thereby reducing resource competition between services and jobs and improving the execution efficiency of batch jobs.

D. RESOURCE ISOLATION MECHANISM

We realize multidimensional resource isolation to minimize interference and improve performance by means of real-time resource monitoring and dynamic resource isolation technology.

1) CPU ISOLATION

The main physical resource shared by services and jobs on the server is the CPU. We should not statically allocate a fixed number of CPUs to services and jobs. If insufficient CPUs are allocated to latency-sensitive services, the SLOs of services can easily be violated. If excessive CPUs are allocated to services, underutilization may occur.

Zeus uses Linux's cpuset CGroups to run services and jobs on different CPUs and dynamically adjusts the configuration of cpuset according to the resource requirements of latency-sensitive services, which not only ensures the CPU resource requirements of services but also allows jobs to fully utilize idle resources of nodes.

When dividing the cpuset, we first sort the hyper-thread according to the NUMA node topology and calculate the size of the cpuset that could be allocated to jobs according to equation (2). As shown in Fig. 7, hyper-threads are divided into two cpusets: the head cpuset is allocated to latency-sensitive services, and the tail cpuset is allocated to batch jobs. When the cpuset of latency-sensitive services changes from large to small, the adjustment step needs to be controlled. Each change can reduce at most 1 physical core. In addition, we direct received network packets of services and jobs to different Rx queues. Packets are sent to different Rx queues and bind to different cores for interrupt processing.

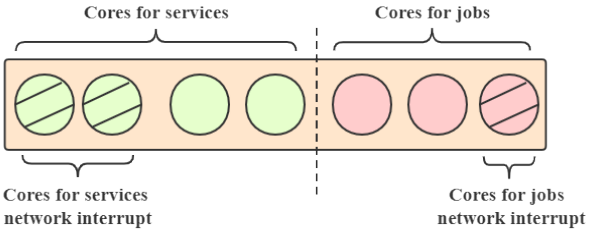


FIGURE 7. Conceptual representation of the cpuset configuration.

2) LLC ISOLATION

Many studies show that cache interference has a substantial impact on service performance [16], [23], [24]. Zeus can run services and jobs on different physical cores through the cpuset, thereby achieving isolation of L1 and L2 Cache. By means of the cache allocation technology (CAT) of Intel CPU hardware, Zeus allocates two independent LLC spaces of different sizes for services and jobs and dynamically adjusts the cache size based on the resource director technology monitoring of CGroups.

3) MEMORY ISOLATION

To prevent batch jobs from draining memory, Zeus limits the upper bound of memory usage of batch jobs through memory CGroups. The calculation of the upper limit is given in equation (2). If the memory used by batch jobs exceeds the upper limit, the job will be killed by CGroups oom.

4) DISK ISOLATION

Spark jobs are IO-intensive; if not restricted, the SLO of latency-sensitive services is easily affected. Zeus dynamically adjusts the direct IO bandwidth of batch jobs through blkio CGroups. On the one hand, this approach can make full use of disk IO bandwidth when services are idle; on the other hand, it can prevent IO contention when services are busy.

5) NETWORK ISOLATION

We configure three VLANs to run control flow, services flow, and batch jobs flow separately. As shown in Fig. 8, the tc htb queue provided by the Linux kernel is used in the egress direction of the server, and bandwidth guarantees and speed limits are implemented for the three VLANs. In addition, servers set different differentiated services code point (DSCP) priorities for the three VLANs. Batch jobs network packets use the lowest priority CS0, services network packets use CS3, and control network packets use the higher priority CS5. When the hardware switch forwards network packets, it trusts the DSCP mark set by servers and preferentially forwards control packets and services packets and discards batch jobs packets first to ensure the network QoS when network congestion occurs.

In the direction of server ingress, we use intermediate functional block (ifb) technology to control the sending speed of the sender through drop packets at the receiver and limit the TCP traffic bandwidth of batch jobs.

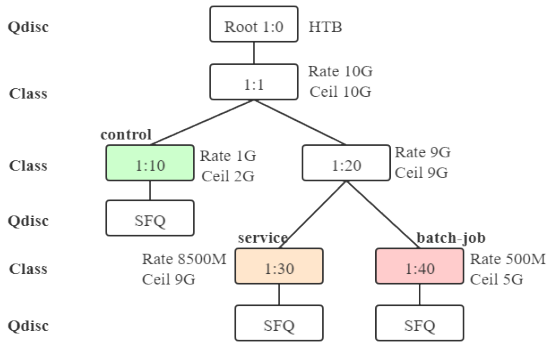


FIGURE 8. Egress traffic control.

Through Intel Ethernet flow director technology, we send received network packets of batch jobs to a separate Rx queue and bind an independent CPU core for interrupt processing. We achieve isolation of batch jobs and latency-sensitive services at the network Rx queue and network interrupt processing level.

As shown in Fig. 9, network isolation is highly effective. We also evaluate the effectiveness with UDP_RR: the results are similar to those with TCP_RR.

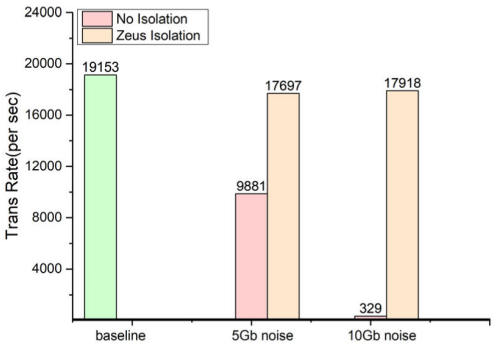


FIGURE 9. TCP_RR metrics under different levels of traffic noise with and without isolation.

IV. EXPERIMENTAL EVALUATION

To fully evaluate and verify the effectiveness of the Zeus system, we first run three representative latency-sensitive services (Nginx, Redis and MySQL) from NetEase on a single machine and then analyze the resource usage and application performance of an over 5000-node production Kubernetes cluster armed with Zeus. CPU resources are more expensive than other physical resources on a server, so we focus on CPU utilization. Tail latency, usually the 99.9^{th} percentile latency, is the main metric used to measure application performance.

The purpose of the experiment:

- Determine how tail latency is impacted when colocating batch jobs with latency-sensitive services without Zeus.
- Assess how effective Zeus is in maintaining tail latency and improving resource efficiency when batch jobs are collocated with latency-sensitive services.
- Obtain reasonable system parameters *tc* through experimental analysis.

A. EXPERIMENTAL SETUP

We use typical production hardware to evaluate the Zeus system. Each Kubernetes node contains two Xeon E5-2680 v4 processors with 14 physical cores per socket (a total of 56 hyper-threads), 256 GB of RAM, 10 Gbit/s Ethernet card, 1 TB HDD for the operating system, 1 TB HDD for the container file system, and 500 GB SSD for batch jobs. The operating system is Debian 10, kernel version 4.19.75.

1) LATENCY-SENSITIVE SERVICES

We select three representative services for the colocation experiments:

- MySQL: A lightweight relational database management system that is latency-sensitive and resource-intensive widely used in Internet services. We use sysbench to simulate user behavior and assess the MySQL performance metrics.
- Redis: A popular high-performance key-value database that is latency-sensitive. We use the memtier benchmark to simulate user read and write operations.
- Nginx: Lightweight web server with high concurrency that is latency-sensitive and widely used in Internet projects. We use the wrk benchmark to simulate user behavior and assess Nginx performance.

2) BATCH JOBS

Apache Spark is an efficient and unified analytics engine for large-scale data processing engines, similar to the Hadoop MapReduce computing framework. However, compared to MapReduce, Spark is efficient in batch processing and has low latency due to its scalability, memory-based computing, and the ability to directly read and write data in any format on Hadoop [25], [26]. Spark has become a lightweight unified platform for rapid processing of big data. Various applications, such as real-time stream processing, machine learning, and interactive query, can be built on different storage and operating systems through Spark. We use TPC-DS to simulate the Spark workload, and the dataset size is 50 GB. TPC-DS is a decision support benchmark launched by the TPC organization to replace TPC-H. It is a resource-intensive SQL test suite for evaluating decision support systems (or data warehouses) in real-world scenarios. This test suite contains complex applications, such as statistics, report generation, online query, and data mining of large datasets [27].

3) SINGLE-MACHINE EXPERIMENT

We collocate the above three latency-sensitive services with TPC-DS and constantly vary the load of TPC-DS and the amount of allocatable colocation resources. Then, we measure the 99.9^{th} percentile tail latency of latency-sensitive services and the throughput of TPC-DS under different resource utilizations. Each experimental condition is tested 10 times, and the average value is taken as the experimental result.

4) PRODUCTION CLUSTER ANALYSIS

We analyze CPU usage and application performance of an over 5000-node production Kubernetes cluster in NetEase. We have collocated latency-sensitive services with batch jobs in the production cluster for more than two years.

B. EXPERIMENTAL RESULTS

1) SINGLE-MACHINE EXPERIMENTS

We measure the resource utilization and tail latency for each of the above services under three deployment scenarios:

- *Baseline:* Latency-sensitive services run alone.
- *No Isolation:* Latency-sensitive services are collocated with TPC-DS jobs in the native Kubernetes cluster. We vary the load of TPC-DS and restrict the CPU utilization of nodes to under 50%, 60%, 70% and 80%.
- *Zeus Isolation:* We collocate TPC-DS jobs and services, configuring different system parameters *tc* and allocating different amounts of resource to jobs. TPC-DS jobs consume as many resources as possible.

We present the comparative results of the tail latency for the above three services.

Fig. 10 shows the Nginx request latency under different deployment scenarios. The baseline tail latency is 0.8 ms. In the native Kubernetes cluster, when Nginx runs together with Spark jobs, its SLOs are easily violated. Then, we enable Zeus and vary the system parameter *tc* to evaluate the effectiveness of the isolation mechanisms. When *tc* is less than 70%, Zeus can ensure a less than 0.1 ms performance degradation for the 99.9^{th} percentile tail latency and satisfy the Nginx SLOs. As shown in Fig. 11, Spark jobs consume as many resources as possible, maximize CPU utilization and make full use of redundant resources that the services require but are currently idle. Moreover, CPU utilization can be increased up to the value specified by system parameter *tc*.

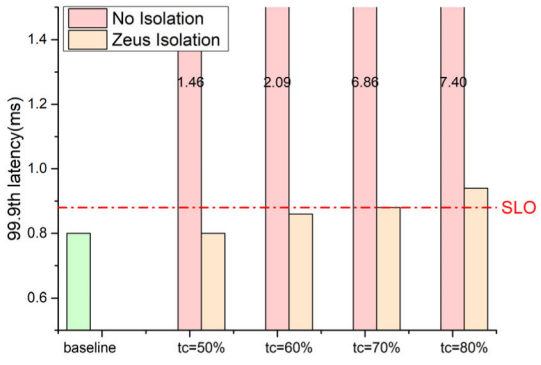


FIGURE 10. The impact on Nginx tail latency when varying Zeus parameter tc.

Fig. 12 reports the measured degradation of Redis tail latency when Redis is collocated with Spark in the native Kubernetes cluster without enhanced isolation. Then, we apply Zeus and vary system parameter *tc* from 50% to 80%. When *tc* is less than 60%, Zeus can minimize

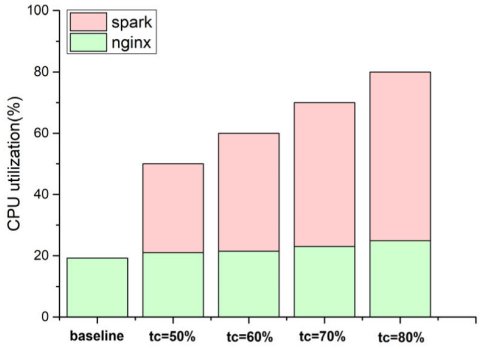


FIGURE 11. The CPU utilization when colocating Nginx with spark under different values of system parameter tc.

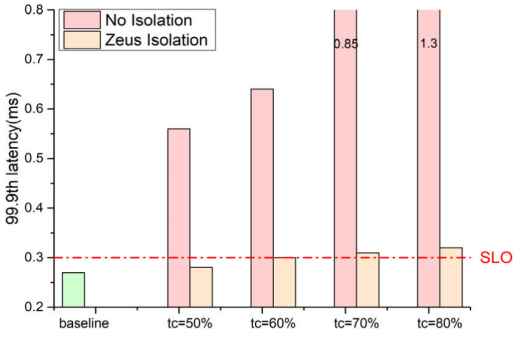


FIGURE 12. The impact on redis tail latency when varying Zeus parameter tc.

the performance interference and meet Redis SLOs. The CPU utilization graph is similar to that in Fig. 11.

Fig. 13 shows the MySQL query latency under different deployment scenarios. The baseline tail latency is 7.8 ms. In the native Kubernetes cluster, when MySQL runs together with Spark jobs, its SLOs are easily violated. Then, we enable Zeus and vary system parameter *tc* to evaluate the effectiveness of the isolation mechanisms. When *tc* is less than 60%, Zeus can effectively avoid performance interference and meet the MySQL SLOs. The CPU utilization graph is similar to that in Fig. 11.

We find that it is safe and effective to set system parameter *tc* to 60%, and the server CPU utilization can be very safely increased to 60% through workload colocation. For CPU resources, we compare the effectiveness of different isolation algorithms when CPU utilization reaches 60%: cpuset is the isolation method used by Zeus, cfs quota is the native CPU isolation capability of Kubernetes, and CPU share can allocate different proportions of CPU time slices to different services. The results shown in Fig. 14 indicate that cpuset is most effective for isolating CPU resources between latency-sensitive services and batch jobs.

Although batch jobs have lower priority and are best-effort, their execution efficiency and throughput are still important. We evaluate the performance of Spark tasks under two deployment scenarios: (1) Spark tasks are deployed as Kubernetes Guaranteed Pods; (2) Spark tasks are run

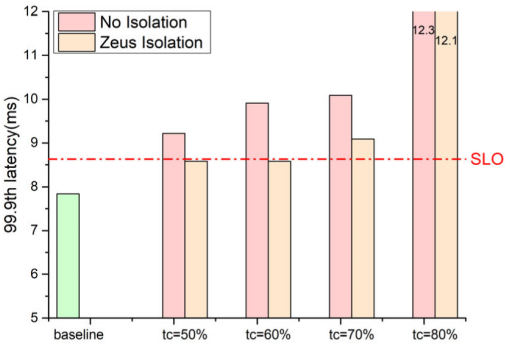


FIGURE 13. The impact on MySQL tail latency when varying Zeus parameter tc.

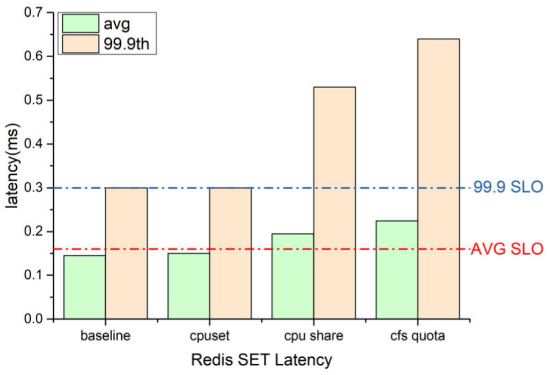


FIGURE 14. Comparison of the effectiveness of different CPU isolation approaches when the CPU utilization reaches 60%.

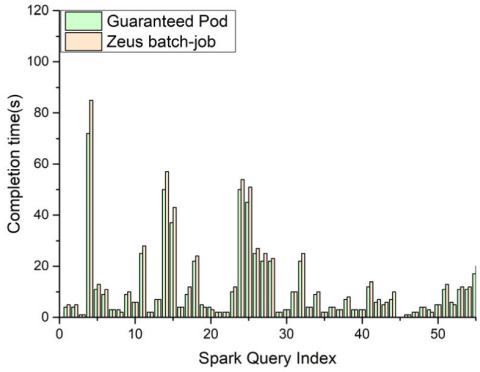


FIGURE 15. Performance comparison of TPC-DS spark under different deployment modes.

as low-priority batch jobs through the Zeus system, using low-quality resources such as colocation/cpu and colocation/memory. As shown in Fig. 15, when Spark runs in low-priority batch jobs mode, the impact on performance does not exceed 20% compared to baseline.

2) PRODUCTION CLUSTER ANALYSIS

The Zeus system has been used stably for more than 2 years in NetEase. We analyze the cluster data of an over 5000-node production Kubernetes cluster to compare the

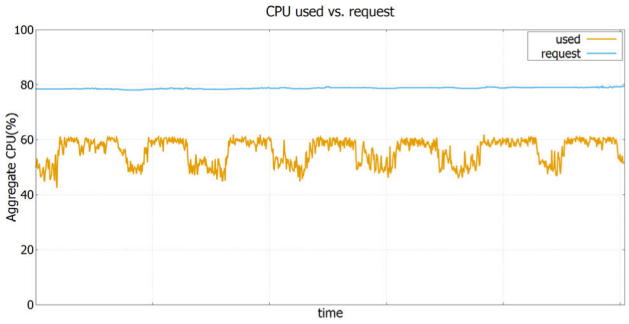


FIGURE 16. After colocation, the CPU utilization of a 5000-node cluster in the production environment for one week.

service performance and resource utilization before and after workload colocation. We collect metrics data for a week.

As shown in Fig. 16, after colocation, the average CPU utilization has increased from 20% to approximately 55%: the resource efficiency has been greatly improved.

The prerequisite for improving resource utilization is not to affect the SLO of latency-sensitive services. We measure the latency of Redis cluster SET and GET operations before and after colocating with Spark batch jobs. As shown in Figs. 17 and 18, the latency of Redis SET and GET operations before and after colocation are essentially unchanged. Furthermore, Tables 3 and 4 show that the Redis GET latency before and after colocation is 0.43 ms and 0.44 ms, respectively, and the performance is reduced by 2.3%. Redis SET latency is 0.59 ms before colocation and 0.62 ms after colocation, and the performance degrades by 5%. Compared

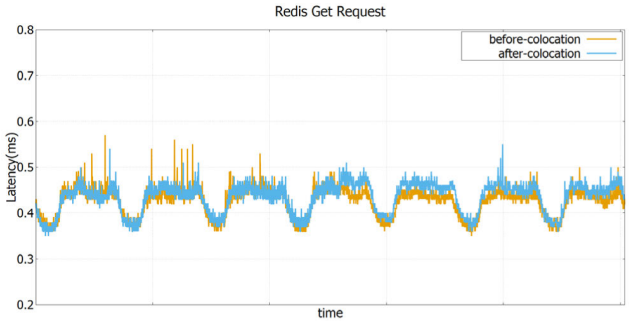


FIGURE 17. Comparison of redis get operation latency in production kubernetes cluster before and after colocation.

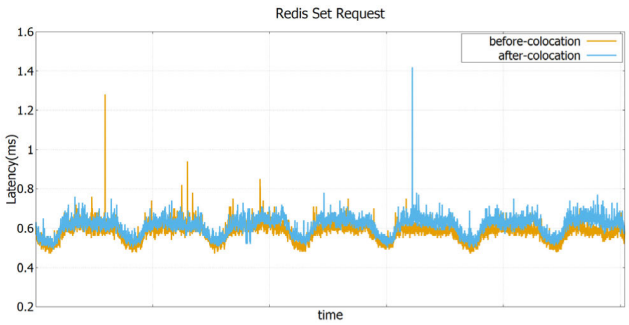


FIGURE 18. Comparison of redis set operation latency in production kubernetes cluster before and after colocation.

TABLE 3. Redis get operation latency before and after colocation.

GET	Number of calls	Average latency	ms0_3
pre-colocation	8700 million	0.43 ms	99.55%
after-colocation	7800 million	0.44 ms	99.50%

TABLE 4. Redis set operation latency before and after colocation.

SET	Number of calls	Average latency	ms0_3
pre-colocation	19 million	0.59 ms	99.47%
after-colocation	18 million	0.62 ms	99.42%

with the resource utilization improvement and cost reduction achieved by workload colocation, such a small performance degradation is negligible.

V. RELATED WORK

Many papers propose various solutions to improve data center resource utilization. However, most of the solutions have limitations: they rely on services to provide SLO and real-time performance metrics API; they rely on specific hardware and OS support; they do not consider the throughput of batch jobs, which leads to lower job efficiency; or they are only applicable to specific scenarios and are not universal. We propose a universal colocation solution for latency-sensitive services and batch jobs, called Zeus, which has multiple technical innovation advantages. We develop Zeus based on the extension mechanism of Kubernetes, do not change any Kubernetes in-tree code, treat services as a black box, and do not rely on special hardware features. Therefore, Zeus can easily be deployed to any standard Kubernetes cluster and has good generalization ability. Moreover, Zeus considers not only the performance of latency-sensitive services but also the throughput of batch jobs. Zeus can increase the average CPU utilization from 15% to 60% without SLO violations.

Bistro [28] is a scheduler for batch jobs that uses a hierarchical scheduling model, but it relies on static allocation of reserved resources for latency-sensitive services, and batch jobs use the remaining fixed-size resources. The static policy is either too conservative (low resource utilization) or overly optimistic (leading to SLO violations). Moreover, Bistro emphasizes scheduling, and does not consider the problem of resource contention between jobs and services on the node. In contrast, Zeus dynamically adjusts the resource allocation, and batch jobs can make full use of the temporarily unused resources of latency-sensitive services to maximize the resource utilization.

PerfIso [4] is a performance isolation framework that runs services/jobs on different CPU cores and dynamically adjusts the CPU cores allocated to jobs according to the load of services. However, PerfIso does not consider the isolation of other physical resources shared by services and jobs, such as caches and networks. Moreover, it is implemented based on the Windows system and uses nonwork-conserving

operating system scheduling algorithms, which limits the wider adoption of this framework. Conversely, Zeus design a multidimensional resource isolation framework by coordinating software and hardware mechanisms to achieve resource isolation on CPU, memory, cache, and network, effectively reducing the performance interference between services and jobs. In addition, Zeus is implemented based on Kubernetes and Linux, which is beneficial to its wide deployment.

Heracles [16] dynamically adjusts isolation parameters based on the real-time latency of services and adjusts the amounts of resources allocated to batch jobs. Heracles requires knowledge of the services SLO and real-time latencies, which limits wide deployment in large-scale production environments. Meanwhile, Zeus regards services as a black box, and does not make assumptions about their implementation. Furthermore, Zeus needs to ensure only that there are sufficient idle resources when services are at the peak load.

Elfen [3] achieves CPU-level isolation through simultaneous multithreading technology. However, Elfen needs services to provide latency and SLO query interfaces, which is impossible or expensive in large-scale production environments. Moreover, Elfen is implemented at the OS kernel level, which is impractical and imposes substantial overhead.

Bubble-Up [2] is concerned mainly with memory bandwidth interference in colocation scenario and does not take other shared resources into consideration. Additionally, it must perform a colocation experiment for every possible colocation scenario to obtain the degree of performance degradation for services under different memory pressure from batch jobs. However, latency-sensitive services in the production environment are complex and diverse, and frequent version upgrades are performed. Additionally, the performance model may change, so this approach is not practical.

Quasar [19] is a resource management system that can improve resource utilization while meeting services SLOs. The system uses collaborative filtering technology to calculate resource requirements and adjusts resource allocations dynamically based on the real-time performance information of the service. In contrast, Zeus does not assume that services provide explicit real-time latency and SLO requirements.

Paragon [15] is an online scalable scheduler based on collaborative filtering technology that considers server heterogeneity and interference between shared resources, quickly classifies and recognizes unknown services, and schedules services to nodes with the best performance. At the same time, Paragon can maximize server utilization. However, it focuses on scheduling issues and does not consider the isolation of shared resources.

ROSE [22] is a resource over-subscription framework. Batch jobs can make full use of the redundant resources that the services require but are currently not using to improve resource utilization while meeting services SLOs. Zeus is similar but does not oversell the resources of services and does not change the QoS semantics of services in Kubernetes, which can fully guarantee the SLO of latency-sensitive services.

Townend *et al.* [29] propose a solution that coordinates software and hardware isolation mechanisms to improve the efficiency of data centers and implement their own scheduler based on Kubernetes. However, its focus is on energy saving, and it does not consider the services SLOs.

VI. CONCLUSION

The resource utilization of data centers is low, which results in high operational costs and substantial resource wastage. Colocating best-effort batch jobs with latency-sensitive services can effectively improve resource efficiency.

This paper presents the design and implementation of Zeus, a cluster scheduling system based on Kubernetes. As an operating system in the cloud computing era, Kubernetes unifies the scheduling of latency-sensitive services and batch jobs. We design a weighted dynamic scheduling algorithm based on a time sliding window and realize multidimensional resource isolation mechanisms to coordinate software and hardware characteristics. Zeus treats services as a black box, does not make any assumptions about services, and can be easily deployed to any standard Kubernetes cluster.

We evaluate Zeus with three representative latency-sensitive services (Redis, MySQL, and Nginx) on a single machine and analyze the resource usage and application performance of an over 5000-node production Kubernetes cluster. The results demonstrate the effectiveness of Zeus for scheduling and resource isolation: Zeus can minimize performance interference and maximize resource utilization. Colocating batch jobs with latency-sensitive services can increase CPU utilization to 60% safely without SLO violations.

In the future, we will improve Zeus in the following aspects: quantifying the interference between latency-sensitive services and batch jobs; reinforcing the isolation of memory bandwidth and buffer IO; and considering the quality of idle resources and expected completion time of batch jobs. Such efforts will enhance the versatility and effectiveness of Zeus in improving resource efficiency.

ACKNOWLEDGMENT

The authors thank Xinyong Wang, Linliang Chen, Jiuyuan Huang, Yingdong Liu, and Chengjie Dai for their help and insight in making our work possible at NetEase.

REFERENCES

[1] U. Hoelzle and L. A. Barroso, *The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines*. San Mateo, CA, USA: Morgan Kaufmann, 2009.

[2] J. Mars, L. Tang, K. Skadron, M. L. Soffa, and R. Hundt, "Increasing utilization in modern warehouse-scale computers using bubble-up," *IEEE Micro*, vol. 32, no. 3, pp. 88–99, May 2012, doi: 10.1109/MM.2012.22.

[3] X. Yang, S. M. Blackburn, and K. S. McKinley, "Elfen scheduling: Fine-grain principled borrowing from latency-critical workloads using simultaneous multithreading," in *Proc. USENIX Conf. Usenix Annu. Tech. Conf.*, Denver, CO, USA, 2016, pp. 309–322.

[4] C. Iorgulescu, R. Azimi, Y. Kwon, S. Elnikety, M. Syamala, V. Narasayya, H. Herodotou, P. Tomita, A. Chen, J. Zhang, and J. Wang, "PerfIso: Performance isolation for commercial latency-sensitive services," in *Proc. USENIX Conf. Usenix Annu. Tech. Conf.*, Boston, MA, USA, 2018, pp. 519–531.
[5] J. M. Kaplan, W. Forrest, and N. Kindler. *Revolutionizing Data Center Energy Efficiency*. Accessed: Apr. 16, 2021. [Online]. Available: https://www.sallan.org/pdfdocs/McKinsey_Data_Center_Efficiency.pdf
[6] C. Reiss, A. Tumanov, G. R. Ganger, R. H. Katz, and M. A. Kozuch, "Heterogeneity and dynamicity of clouds at scale: Google trace analysis," in *Proc. 3rd ACM Symp. Cloud Comput. (SoCC)*, 2012, p. 7.
[7] V. Medel, O. Rana, J. Bañares, and U. Arronategui, "Modelling performance & resource management in Kubernetes," in *Proc. IEEE/ACM 9th Int. Conf. Utility Cloud Comput. (UCC)*, Shanghai, China, Dec. 2016, pp. 257–262.
[8] Z. Zhong and R. Buyya, "A cost-efficient container orchestration strategy in Kubernetes-based cloud computing infrastructures with heterogeneous resources," *ACM Trans. Internet Technol.*, vol. 20, no. 2, pp. 1–24, May 2020, doi: 10.1145/3378447.
[9] D. Bernstein, "Containers and cloud: From LXC to Docker to Kubernetes," *IEEE Cloud Comput.*, vol. 1, no. 3, pp. 81–84, Sep. 2014, doi: 10.1109/MCC.2014.51.
[10] S. Soltesz, H. Po'tzl, M. E. Fiuczynski, A. Bavier, and L. Peterson, "Container-based operating system virtualization: A scalable, high-performance alternative to hypervisors," *ACM SIGOPS Oper. Syst. Rev.*, vol. 41, no. 3, pp. 275–287, 2007, doi: 10.1145/1272998.1273025.
[11] Kubernetes. *Production-Grade Container Orchestration*. Accessed: Apr. 16, 2021. [Online]. Available: https://kubernetes.io/
[12] A. Spart. *Running Spark on Kubernetes*. Accessed: Apr. 16, 2021. [Online]. Available: https://spark.apache.org/docs/latest/running-on-kubernetes.html
[13] D. Bannon, E. Moen, M. Schwartz, E. Borba, S. Cui, K. Huang, I. Camplisson, N. Koe, D. Kyme, T. Kudo, and B. Chang, "Dynamic allocation of computational resources for deep learning-enabled cellular image analysis with Kubernetes," *BioRxiv*, Sep. 2019, Art. no. 505032, doi: 10.1101/505032.
[14] E. Kristiani, C.-T. Yang, and C.-Y. Huang, "ISEC: An optimized deep learning model for image classification on edge computing," *IEEE Access*, vol. 8, pp. 27267–27276, Feb. 2020, doi: 10.1109/ACCESS.2020.2971566.
[15] C. Delimitrou and C. Kozyrakis, "Paragon: QoS-aware scheduling for heterogeneous datacenters," in *Proc. 18th Int. Conf. Archit. Support Program. Lang. Operating Syst.*, Houston, TX, USA, 2013, pp. 77–88.
[16] D. Lo, L. Cheng, R. Govindaraju, P. Ranganathan, and C. Kozyrakis, "Heracles: Improving resource efficiency at scale," in *Proc. 42nd Annu. Int. Symp. Comput. Archit.*, Jun. 2015, pp. 450–462.
[17] S. Xue, C. Hu, J. Zhu, and R. Yang, "Shaready: A resource-IsolatedWorkload co-location system," in *Proc. IEEE Int. Conf. Service-Oriented Syst. Eng. (SOSE)*, Apr. 2019, pp. 299–2995.
[18] B. Cai and K. Li, "SLO-aware colocation: Harvesting transient resources from latency-critical services," *J. Syst. Archit.*, vol. 101, Dec. 2019, Art. no. 101663, doi: 10.1016/j.sysarc.2019.101663.
[19] C. Delimitrou and C. Kozyrakis, "Quasar: Resource-efficient and QoS-aware cluster management," in *Proc. 19th Int Conf. Archit. Support Program. Lang. Operating Syst.*, Salt Lake City, UT, USA, 2014, pp. 127–144.
[20] V. Medel, O. Rana, J. Á. Bañares, and U. Arronategui, "Adaptive application scheduling under interference in Kubernetes," in *Proc. 9th Int. Conf. Utility Cloud Comput.*, Dec. 2016, pp. 426–427.
[21] Y. Li, J. Zhang, C. Jiang, J. Wan, and Z. Ren, "PINE: Optimizing performance isolation in container environments," *IEEE Access*, vol. 7, pp. 30410–30422, 2019, doi: 10.1109/ACCESS.2019.2900451.
[22] X. Zhang, E. Tune, R. Hagmann, R. Jnagal, V. Gokhale, and J. Wilkes, "CPI^2: CPU performance isolation for shared compute clusters," in *Proc. 8th ACM Eur. Conf. Comput. Syst. (EuroSys)*, 2013, pp. 379–391.
[23] Y. Xiang, C. Ye, X. Wang, Y. Luo, and Z. Wang, "EMBA: Efficient memory bandwidth allocation to improve performance on Intel commodity processor," in *Proc. 48th Int. Conf. Parallel Process.*, Aug. 2019, p. 16.
[24] A. Tzenetopoulos, "Interference-aware container orchestration in Kubernetes cluster," M.S. thesis, Dept. Microprocessors Digit. Syst. Lab., Nat. Tech. Univ. Athens, Athens, Greece, 2020.
[25] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, and A. Ghodsi, "Apache spark: A unified engine for big data processing," *Commun. ACM*, vol. 59, no. 11, pp. 56–65, 2016, doi: 10.1145/2934664.
[26] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster computing with working sets," in *Proc. 2nd USENIX Conf. Hot Topics Cloud Comput.*, Boston, MA, USA, 2010, p. 10.
[27] L. Shi, B. Huang, H. Xu, and X.-J. Ye, "Implementation of TPC-DS testing tool," in *Proc. 2nd Int. Workshop Database Technol. Appl.*, Nov. 2010, pp. 1–4.
[28] A. Goder, A. Spiridonov, and Y. Wang, "Bistro: Scheduling data-parallel jobs against live production systems," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2015, pp. 459–471.
[29] P. Townend, S. Clement, D. Burdett, R. Yang, J. Shaw, B. Slater, and J. Xu, "Invited paper: Improving data center efficiency through holistic scheduling in Kubernetes," in *Proc. IEEE Int. Conf. Service-Oriented Syst. Eng. (SOSE)*, Apr. 2019, pp. 156–15610.



XIAOLONG ZHANG received the B.Sc. and Ph.D. degrees in computer science from Zhejiang University. Since 2012, he has been working at NetEase, Inc., where he currently leads an Engineer Research and Development Team to build the NetEase cloud infrastructure. His research interests include distributed systems, cloud computing, and cloud native technologies.



LANQING LI received the B.Sc. degree in computer science and technology from Xi'an Jiaotong University, in 2013, and the M.S. degree in computer application from Zhejiang University, in 2016. He is currently a Senior Software Engineer with NetEase (Hangzhou) Network Company Ltd. His research interests include cloud computing, cloud native, and big data.



YUAN WANG received the B.Sc. and Ph.D. degrees in computer science from Zhejiang University. He is currently the Vice President of NetEase and the Executive Director of Hangzhou Research Institute. He is responsible for cutting-edge technology research, technical platform construction, and government and enterprise business of NetEase ShuFan. His research interests include cloud computing, databases, big data, artificial intelligence, and information safety.



E. CHEN received the B.Sc. and M.S. degrees in information and electronic engineering from Zhejiang University. Since 2006, he has been working at NetEase, Inc. He is currently a Leader of Cloud Computing Department, NetEase. His research interests include distributed systems, cloud computing, cloud native technologies, and low code development.



LIDAN SHOU (Member, IEEE) received the Ph.D. degree in computer science from the National University of Singapore. He is currently a Professor with the College of Computer Science, Zhejiang University, China. He has published more than 100 articles in refereed journals and conference proceedings. His research interests include database systems, information extraction, and web data mining. He is a member of ACM and a Senior Member of CCF.